

Developmental Attention Windows Improve Transformer Training: Forced Specialization Through Layer-Wise Locality Constraints

Nicolás Veraz
Independent Researcher
github.com/nicoveraz/neurogen

Abstract

We investigate whether biologically-inspired developmental principles can improve transformer training. Across 200+ controlled experiments at two scales (3.4M and 125M parameters), we systematically test cellular automata weight initialization, live developmental rules, embryogenic development, pre-wired circuits, and architectural constraints inspired by cortical development. Most approaches produce marginal gains (+0.6%) or actively hurt (−3% to −5%). However, embedding a developmental constraint into the architecture—attention windows that grow from local to global across layers—yields a small but consistent improvement at 3.4M and a larger but noisier (and unconverged) advantage at 125M. At 3.4M the gain is +1.5% across 5 *matched* seeds (all 5 beat their paired baseline); because the runs are paired, the exact one-sided sign-flip permutation test floors at $p = 1/32 = 0.031$, with a parametric paired- t $p \approx 0.001$ and paired effect size $d_z \approx 3.6$. The advantage persists at +0.97% through 100k extended training steps. At 125M the gain is +2.6% averaged across 5 seeds at 20k (one seed negative), widening to +12.9% on the two seeds extended to 50k—but neither model is converged at 50k, so the 50k figure is suggestive rather than a converged scaling result. Seven mechanism experiments cleanly eliminate five hypotheses (gradient noise removal, softmax coupling, variance reduction, landscape smoothness, ongoing structural constraint); a sixth, implicit regularization, is inconclusive (the train-val gap is seed-contradictory at $n = 2$). The data identify the mechanism as a curriculum effect with lasting structural impact: early local-attention constraints create a compositional hierarchy via reduced parameter coupling, and this hierarchy persists after windows are removed. Attention entropy measurements at both 20k and 100k steps confirm the specialization is permanent: early layers maintain ~52% lower entropy at 100k steps, with the gap narrowing only slightly from the 57% measured at 20k.

1 Introduction

Standard transformers start from random initialization with uniform, fully-global attention at every layer. Gradient descent discovers all internal structure—specialized attention heads, hierarchical layer roles, induction circuits—from scratch.

The brain takes a different approach. Genetic developmental programs grow functional structure before learning begins: cortical receptive fields develop local-to-global through layers, induction-like circuits (pattern completion) are pre-wired, and architectural constraints (laminar structure, lateral inhibition, critical periods) protect these scaffolds during early learning.

We ask: can these developmental principles improve transformer training?

Our answer is nuanced. Most biological inspirations fail when applied to transformers. But one specific intervention works consistently: constraining attention to grow from local to global across layers, mimicking cortical receptive field development. At 3.4M parameters the quartic window schedule gives a +1.5% improvement in which all 5 matched seeds beat their paired baseline; combining it with pre-wired induction head circuits does not help further (+1.1%).

At 125M parameters the schedule gives +2.6% across 5 seeds at 20k steps; on the two seeds extended to 50k the gap widens to +12.9%, though neither model is converged there.

Extended training to 100k steps at 3.4M parameters confirms this is not merely faster convergence: the advantage peaks at +2.9% during early training (10k steps), consistent with a curriculum effect, and persists at +1.0% through 100k steps. The mechanism operates through *forced architectural specialization*, not gradient noise removal. Seven targeted experiments demonstrate that attention windows increase gradient signal coherence $18\times$ without reducing gradient noise, dramatically reduce parameter coupling, and create permanent attention hierarchy that persists even after the windows are removed.

2 Related Work

Neural Cellular Automata. Growing Neural Cellular Automata [Mordvintsev et al., 2020] demonstrated self-organizing pattern generation. HyperNCA [Najarro and Risi, 2022] extended this to grow RL policy weights via NCA. Our work tests NCA-inspired approaches for transformer weight initialization and live training dynamics.

Structured Initialization. Mimetic initialization [Trockman and Kolter, 2023] analytically constructs attention patterns ($QK^\top \approx I$). T-Fixup [Huang et al., 2020] uses Lipschitz-constrained initialization. μ P [Yang et al., 2022] provides principled parameterization for scaling. Our approach uses architectural constraints rather than analytical weight formulas.

Universal Circuits. Induction heads [Olsson et al., 2022] are universal match-and-copy circuits found across trained transformers. Circuit tracing [Anthropic, 2025] reveals consistent internal structure. Our work attempts to pre-build these circuits and finds they require architectural protection to survive early training.

Efficient Attention. Sliding window attention [Beltagy et al., 2020] reduces computation for long sequences. Flash Attention [Dao, 2023] provides hardware-efficient attention with native sliding window support. Our contribution is framing layer-wise window growth as a *developmental* constraint that improves training quality, not just efficiency.

3 Method

3.1 Experimental Framework

We follow the autoresearch paradigm [Karpathy, 2026]: an autonomous agent modifies training code, runs fixed-budget experiments, keeps improvements, and discards failures. Over 200 experiments were conducted across two model scales.

3.4M model: GPT-style transformer, depth 4, channels 256, 4 heads. Training data: TinyStories [Eldan and Li, 2023]. Hardware: Apple M1 Pro (MPS). Metric: validation bits-per-byte (val_bpb).

125M model: GPT-2 Small architecture (12 layers, 768 dim, 12 heads, 1024 context). Training data: FineWeb-Edu [Penedo et al., 2024]. Hardware: NVIDIA H100 80GB. Uses RoPE, RMSNorm, bfloat16 mixed precision, and Flash Attention 2 with native sliding window support.

3.2 Developmental Attention Windows

Each layer l (of L total) has a maximum attention window determined by a power function:

$$w(l) = \min\left(T, b + \left(\frac{l+1}{L}\right)^\gamma (T-b)\right) \quad (1)$$

where T is the sequence length, b is the base window size (8 for 3.4M, 16 for 125M), and γ is the growth exponent. Position i can attend to positions $[\max(0, i - w(l) + 1), i]$.

For $\gamma = 4$ (quartic) at depth 4, the window schedule is [8, 10, 65, 256]—early layers see only local context while the final layer has full attention. We tested exponents from 0.5 to 12.0 along with sigmoid, logarithmic, exponential, and Fibonacci schedules. Quartic ($\gamma = 3$ –4) consistently performs best.

Implementation: The window is applied as a causal attention mask. At 3.4M scale, this uses standard masked attention. At 125M scale, we use Flash Attention 2’s native `window_size` parameter, which provides sliding window attention within the fused kernel—windowed models run *faster* than baseline (2.84 vs 2.78 steps/sec).

3.3 Approaches Tested

We organized 26 approaches into four categories:

Category A: Weight Decoration (modify weights, preserve architecture). CA initialization (grid, block-diagonal, reaction-diffusion, spectral); live CA during training (homeostatic, competition, pruning); embryogenic CA (activity-dependent rules using gradients during critical period); pre-wired circuits (induction heads, layer-role differentiation).

Category B: Architectural Constraints (change how the model computes). Attention window growth schedules (linear, quadratic, quartic, higher powers); CA-generated attention bias.

Category C: Combinations. Windows + induction pre-wiring; windows + embryogenic CA; all combined.

Category D: Radical modifications. CA modulation channels; token vitality (cell death); sleep consolidation; cross-layer CA state; developmental dropout.

4 Results

4.1 3.4M Validation (20k steps, 5 seeds)

Table 1 shows the validated results at 3.4M parameters. The runs are *paired*—each seed trains the baseline and the window variant from the same initialization—so we analyze the per-seed differences with a paired test rather than the unpaired Welch test used in an earlier draft (which also used a normal approximation in place of a t -distribution and thereby overstated significance). For every window variant all 5 paired differences are positive, so the exact one-sided sign-flip permutation test reaches its floor of $1/2^5 = 1/32 = 0.031$; the parametric paired- t and the paired effect size d_z (mean difference over its standard deviation) separate the variants. The paired residual standard deviation for the quartic variant (≈ 0.004 bpb) is roughly 14× smaller than the run-to-run MPS noise floor (≈ 0.055 bpb), which is why a sub-noise-floor mean effect is detectable. Throughput is identical across all architectures (4.8 steps/sec on M1 Pro).

Table 1: 3.4M validation results (20,000 steps, 5 matched seeds). p_{perm} is the exact one-sided sign-flip permutation p over the 5 paired differences (floor 1/32); p_t is the two-sided paired t -test; d_z is the paired effect size.

Config	n	Mean bpb	Std	vs Baseline	p_{perm}	p_t	d_z
Baseline	5	0.9002	0.0075	—	—	—	—
Quadratic ($\gamma = 2$)	5	0.8911	0.0048	+1.0%	0.031	0.012	1.94
Quartic ($\gamma = 4$)	5	0.8866	0.0056	+1.5%	0.031	0.001	3.59
Quad + Induction	5	0.8899	0.0041	+1.1%	0.031	0.009	2.09

4.2 What Failed

Table 2 summarizes the negative results. The transformer resists weight decoration—adding structure to weights produces marginal gains at best and frequently causes model collapse.

Table 2: Summary of unsuccessful approaches.

Approach	Best result	Experiments	Failure mode
CA initialization	+0.6%	40+	Marginal; simpler patterns > complex
Live CA	−2 to −5%	15+	Per-step overhead kills performance
Embryogenic CA	+1.3%	16	Modest; high overhead
Pre-wired circuits alone	−3 to −5%	12	Gradients destroy pre-wired structure
CA modulation channels	divergence	8	Model collapse
Token vitality	divergence	4	Model collapse
Sleep consolidation	−1%	6	Overhead > benefit

4.3 125M Scaling (Preliminary)

At 125M parameters the advantage is real but smaller and noisier than the 3.4M result, and the strongest-looking number comes from unconverged runs. Two cuts of the data make this clear. First, across all 5 seeds at a matched 20k steps (Table 3), the mean gap is +2.6% (sd 1.9), and one of the 5 seeds is *negative*—the quartic model is worse on seed 256. Each seed’s baseline and quartic arms share an LR schedule, so the within-seed gap is valid even though seeds 42/137 run a 50k schedule (read at step 20k) and 256/789/1337 are dedicated 20k runs.

Table 3: 125M per-seed gap at a matched 20k steps (all 5 seeds).

Seed	Baseline bpb	Quartic bpb	Gap
42	4.101	3.894	+5.04%
137	4.090	3.989	+2.47%
256	3.696	3.704	−0.21%
789	3.578	3.503	+2.09%
1337	3.620	3.492	+3.53%
Mean	—	—	+ 2.6%

Second, on the two seeds extended to 50k (Table 4) the gap widens to +12.9% (gap of means; per-seed +17.2% and +8.4%). But *neither model is converged*—both baselines are still descending ~ 0.01 bpb per 1k steps at step 50k—so a still-widening gap is exactly what an unconverged head-start produces. We therefore report the 50k figure as suggestive, not as a converged scaling result. A fresh from-scratch comparison under a single fully-annealed horizon is the experiment that would distinguish a genuine scaling effect from an undertraining artifact.

Table 4: 125M gap evolution on the two seeds extended to 50k (mean of seeds 42 and 137). Both models are unconverged at 50k; read as suggestive.

Steps	Baseline bpb	Quartic bpb	Gap
5k	4.911	4.820	+1.9%
10k	4.580	4.390	+4.2%
20k	4.095	3.942	+3.8%
30k	3.844	3.576	+7.0%
40k	3.538	3.188	+9.9%
50k	3.447	3.001	+12.9%

Windowed models run slightly faster than baseline (2.84 vs 2.78 steps/sec) due to Flash Attention’s sliding window computing fewer attention scores.

4.4 3.4M Extended Training (100k steps)

To confirm the advantage is not merely faster early convergence, we trained both baseline and quartic models for 100k steps (seed 42). Table 5 shows the quartic advantage peaks early and persists through $5\times$ longer training.

Table 5: 3.4M gap evolution over 100k steps (seed 42).

Steps	Baseline bpb	Quartic bpb	Gap	Note
1k	1.339	1.326	+1.0%	
5k	1.122	1.109	+1.2%	
10k	1.051	1.021	+2.9%	peak gap
20k	0.971	0.957	+1.4%	
50k	0.898	0.893	+0.6%	
100k	0.807	0.799	+1.0%	persists

The curriculum effect is strongest during early training (peak +2.9% at 10k steps), consistent with the hypothesis that windows shape hierarchy formation in the critical early phase. The advantage narrows during mid-training but never closes, stabilizing at $\sim +1\%$ through 100k steps. Effective attention spans at 100k steps confirm that quartic early layers develop genuine locality: layer 0 attends to only ~ 2 of its 8 allowed tokens, while baseline layer 0 attends to ~ 8 of 256.

5 Mechanism Analysis

We conducted seven experiments to identify why attention windows improve training.

5.1 Experiment 1: Gradient Quality vs Window Size

On a frozen trained 3.4M checkpoint, we measured gradient properties at 10 window sizes (8–256 tokens, 50 backward passes each, layer 0 only).

Key finding: Gradient noise norm is *constant* (~ 0.0053) across all window sizes. What changes is signal norm—it increases $18\times$ from window 256 (0.0017) to window 8 (0.032). The gradient signal-to-noise ratio (SNR) shows a knee at ~ 32 –48 tokens, matching the model’s natural attention span (Table 6).

Table 6: Gradient properties vs window size (layer 0, 50 passes).

Window	SNR	Signal norm	Noise norm	Stability
8	5.588	0.03227	0.00578	0.973
16	3.226	0.01745	0.00541	0.919
32	0.773	0.00414	0.00536	0.363
64	0.343	0.00181	0.00529	0.089
128	0.337	0.00183	0.00544	0.087
256	0.328	0.00173	0.00527	0.080

Windows increase gradient *coherence* (signal), not reduce noise. This rules out gradient noise removal as the mechanism.

5.2 Experiment 2: Gradient Decomposition

We decomposed the softmax backward pass into contributions from attended ($\alpha_{ij} > 1/T$) vs non-attended positions across all 4 layers.

Noise fraction is only 4–7% across all layers (Table 7). The softmax coupling introduces minimal gradient contamination, eliminating the “gradient noise removal” hypothesis.

Table 7: Gradient decomposition by layer.

Layer	Noise fraction	Entropy	Mean span	Attended positions
0	$5.0 \pm 0.03\%$	1.988	12.9	17.3
1	$4.0 \pm 0.02\%$	1.500	7.4	10.9
2	$6.9 \pm 0.03\%$	2.453	21.3	23.3
3	$6.7 \pm 0.05\%$	2.110	13.0	18.7

5.3 Experiment 3: Variance Reduction Control

If windows improve training through gradient variance reduction, then larger batch sizes (which also reduce variance) should replicate the effect. We compared four configurations at 2000 training steps:

Table 8: Variance reduction comparison (mean of 3 seeds, equal optimizer steps).

Config	Eff. batch	bpb	Tokens	bpb at 16M tokens
A: Full, batch 32	32	1.244	16M	1.244
B: Quartic, batch 32	32	1.214	16M	1.214
C: Full, batch 128	128	1.084	66M	~ 1.352
D: Full, batch 256	256	1.033	131M	~ 1.45

At equal token count (16M), quartic windows (1.214) outperform both baseline (1.244) and the larger-batch controls (~ 1.352 , ~ 1.45). Variance reduction cannot replicate the window effect. The mechanism is specific to attention restriction.

5.4 Experiment 4: Train-Val Gap (Implicit Regularization)

If windows act as implicit regularizers, they should reduce overfitting—quartic models should have a *smaller* train-val gap than baseline. We measured the training and validation bpb of

the frozen baseline and quartic checkpoints to compute the generalization gap on both available seeds.

Key finding (inconclusive at $n = 2$): The two seeds disagree. On seed 42 the quartic model has a *larger* gap than baseline (+2.48% vs +0.32%), consistent with “not regularization.” On seed 137 the quartic model has a *smaller* (in fact negative) gap (−2.70% vs +0.80%), the opposite direction, consistent with regularization. With only two seeds and contradictory signs, this experiment does not cleanly rule implicit regularization in or out; we therefore do not count it among the eliminated hypotheses. (An earlier draft reported only the seed-42 direction and a clean elimination.)

5.5 Experiment 5: Gradient Covariance Rank (Parameter Coupling)

We measured the effective rank of the gradient covariance matrix under different window sizes to test whether windows reduce parameter coupling. On a frozen baseline checkpoint, we collected 50 gradient samples for the Q/K weights of layer 0 at five window sizes and computed the SVD of the gradient matrix (Table 9).

Table 9: Gradient covariance rank vs window size (layer 0, Q/K weights, 50 passes).

Window	Effective rank	Var. top-1	Var. top-5
8	17.2	96.7%	97.3%
32	45.5	34.0%	45.3%
64	48.4	9.2%	25.2%
128	48.6	5.9%	23.0%
256	48.6	6.1%	22.8%

Key finding: Effective rank drops from 48.6 (full attention) to 17.2 (window 8)—a $2.8\times$ reduction. At window 8, 96.7% of gradient variance concentrates in the top singular component, versus 6.1% at window 256. Windows *dramatically reduce parameter coupling* at early layers, forcing each gradient step to update a coherent, low-dimensional subspace rather than diffuse, high-dimensional noise. This is the one surviving mechanism. (Numbers from `experiment_mechanism.py -exp5`, committed in `gradient_results/mechanism_disambiguation.json`.)

5.6 Experiment 6: Gradient Stability (Landscape Smoothness)

If windows produce a smoother optimization landscape, then models *trained with* quartic windows should exhibit more stable gradients at convergence—even when evaluated with standard (unmasked) forward passes. We compared gradient direction stability (mean pairwise cosine similarity across 30 gradient samples) for baseline vs quartic checkpoints on both available seeds.

Key finding: Quartic-trained models have slightly *lower* gradient stability than baseline on *both* seeds (mean 0.072 vs 0.074; seed 42: 0.0335 vs 0.0358; seed 137: 0.110 vs 0.112). The quartic model finds a *better* solution, but not one in a smoother region of the loss landscape. Optimization landscape smoothness is eliminated as a mechanism.

5.7 Experiment 7: Remove Windows Mid-Training (Curriculum vs Structure)

This experiment distinguishes three sub-hypotheses about the nature of the window effect:

- **Structural specialization:** Windows create permanent internal hierarchy. Removing them should preserve the benefit ($F \approx B$).

- **Ongoing constraint:** Windows must remain active to maintain the benefit. Removing them should degrade performance (F worse than B).
- **Curriculum effect:** Windows bootstrap useful structure but eventually constrain. Removing them should *improve* performance (F better than B).

We trained three configurations: A (full attention, 20k steps), B (quartic windows, 20k steps), and F (quartic for 10k steps, then full attention for 10k steps), across 3 seeds (Table 10).

Table 10: Curriculum test: removing windows mid-training (mean of 2 seeds*).

Config	Steps with windows	Final bpb	vs Baseline
A: Full attention	0 / 20k	0.8977	—
B: Quartic windows	20k / 20k	0.8858	+1.33%
F: Quartic $\rightarrow$ Full	10k / 20k	0.8842	+1.50%

*Seed 256 Config F diverged after window removal (NaN); results from seeds 42 and 137 only.

Key finding: $F > B > A$. Removing windows after 10k steps actually finds a *better* solution ($F = 0.8842$) than keeping them for all 20k steps ($B = 0.8858$). This supports a **curriculum effect with lasting structural impact**: early constraints create a compositional hierarchy that persists after removal, and permanent windows eventually become a mild constraint on the upper layers. The ongoing structural constraint hypothesis is eliminated.

5.8 Attention Entropy Confirmation

As independent confirmation of forced specialization, we measured per-layer attention entropy on validation data (100 batches) for both baseline and quartic checkpoints. For each layer, we computed entropy $H = -\sum_j \alpha_{ij} \log \alpha_{ij}$ averaged over all heads, query positions, and batches. To test whether this specialization is permanent or transient, we measured entropy at both 20k and 100k training steps (Table 11).

Table 11: Attention entropy per layer at 20k and 100k steps (seed 42, 100 batches).

Layer	20k steps			100k steps		
	Baseline	Quartic	Change	Baseline	Quartic	Change
0	1.994	0.852	−57.3%	1.860	0.898	−51.7%
1	1.506	0.916	−39.2%	1.933	1.248	−35.4%
2	2.466	2.183	−11.5%	2.722	2.489	−8.6%
3	2.120	2.449	+15.5%	2.382	2.605	+9.4%

The specialization is *permanent*. At 100k steps—5 $\times$ longer training—quartic models still show 52% lower entropy at layer 0 and 35% lower at layer 1. The entropy gap narrows only slightly from 20k to 100k, confirming that the developmental hierarchy is not a transient artifact of early training but a lasting structural feature. The final layer consistently shows slightly *higher* entropy in the quartic model, compensating by using its full attention span to integrate globally over the local features built by the constrained early layers.

5.9 Mechanism Summary

The data converges on a **curriculum effect with lasting structural impact**. Windows during early training force a local-to-global learning order that creates a compositional hierarchy.

This hierarchy persists after windows are removed—the model does not unlearn it. Notably, removing windows after 10k steps actually finds a *better* solution ($F = 0.8842$) than keeping them for all 20k steps ($B = 0.8858$), suggesting that permanent windows eventually become a mild constraint on the upper layers. The reduced parameter coupling (Experiment 5) is the likely mechanism by which early constraints shape the hierarchy: with fewer interacting parameters per gradient step, early layers are forced to find coherent local solutions rather than diffuse global patterns. The attention entropy analysis independently confirms that this specialization is reflected in the learned attention patterns: early layers develop focused, low-entropy attention while the final layer maintains broader coverage.

6 Limitations

- 125M results are preliminary and noisy: the 20k comparison spans 5 seeds (mean +2.6%) but one seed is negative, and the +12.9% figure rests on only 2 seeds extended to 50k, where neither model is converged. We have not run a fresh from-scratch comparison under a single fully-annealed horizon, which is what would distinguish a real scaling effect from an undertraining artifact.
- Implicit regularization (Experiment 4) is not resolved: the train–val gap is seed-contradictory at $n = 2$.
- No downstream task evaluation (CORE, MMLU, etc.).
- The 3.4M model uses TinyStories (narrow domain); 125M uses FineWeb-Edu (broader).
- Optimal exponent at depth 12 not yet determined (tested only quartic).
- Mechanism experiments performed only at 3.4M scale.
- Experiment 7 has only 2 valid seeds (seed 256 diverged after window removal).

7 Discussion

Scaling. The effect appears to grow with scale (+1.5% at 3.4M $\rightarrow$ +2.6% across 5 seeds at 125M/20k, $\rightarrow$ +12.9% on 2 unconverged seeds at 50k). The trend is consistent with more layers providing a richer developmental progression, but the 125M evidence is not yet strong enough to assert a converged scaling law; we flag this as the most important open experiment.

Practical implications. Two deployment strategies emerge from Experiment 7. Either keep windows permanently (simple, slight throughput bonus from Flash Attention sliding window) or apply windows only during the first half of training then remove them (allows full attention at inference with a marginally better result).

Biological connection. The curriculum interpretation aligns with critical periods in cortical development: constrained connectivity during early development shapes functional architecture that persists after the constraints relax. The quartic growth exponent creates a steep developmental curve where most layers stay very local, with only the final layers opening to full context.

8 Conclusion

Across 200+ experiments at two scales, we find that developmental attention windows—constraining early layers to local attention with quartic growth toward full attention—improve transformer training: +1.5% at 3.4M with all 5 matched seeds beating their paired baseline (exact paired

permutation $p = 1/32 = 0.031$, paired- t $p \approx 0.001$), persisting at +1.0% through 100k extended training, and +2.6% across 5 seeds at 125M/20k (one seed negative; widening to +12.9% on 2 unconverged seeds at 50k). Seven mechanism experiments cleanly eliminate five hypotheses (a sixth, implicit regularization, is inconclusive) and identify the operative mechanism as a **curriculum effect with lasting structural impact**: early local-attention constraints create a compositional hierarchy via reduced parameter coupling, and this hierarchy persists permanently—confirmed by attention entropy measurements showing 52% lower entropy at early layers even at $5\times$ longer training. The approach requires no additional parameters, no extra compute, and can be applied either permanently or only during early training.

Code and data: <https://github.com/nicoveraz/neurogen>

References

- I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The Long-Document Transformer. *arXiv:2004.05150*, 2020.
- T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv:2307.08691*, 2023.
- R. Eldan and Y. Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? *arXiv:2305.07759*, 2023.
- J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training Compute-Optimal Large Language Models. *arXiv:2203.15556*, 2022.
- X. S. Huang, F. Perez, J. Ba, and M. Volkovs. Improving Transformer Optimization Through Better Initialization. *ICML*, 2020.
- A. Karpathy. nanochat / autoresearch. <https://github.com/karpathy/autoresearch>, 2026.
- A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing Neural Cellular Automata. *Distill*, 2020.
- E. Najarro and S. Risi. HyperNCA: Growing Neural Cellular Automata to Grow Neural Networks. *arXiv:2204.11674*, 2022.
- C. Olsson, N. Elhage, N. Nanda, et al. In-context Learning and Induction Heads. *Transformer Circuits Thread*, 2022.
- G. Penedo, H. Malartic, D. Hesslow, et al. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *arXiv:2406.17557*, 2024.
- A. Trockman and J. Z. Kolter. Mimetic Initialization of Self-Attention Layers. *arXiv:2305.09828*, 2023.
- G. Yang, E. J. Hu, I. Babuschkin, et al. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. *arXiv:2203.03466*, 2022.
- Anthropic. Circuit Tracing: Revealing Computational Graphs in Language Models. *Transformer Circuits Thread*, 2025.